

Computer

Course code:COMP-203

Solved Past Papers

Past paper:2019

Question no 1

What is compiling?

Definition:

Compiling is the process of converting human-readable source code into machine code that a computer can execute. This process is done by a **compiler**, a special program that translates high-level programming languages (like C, C++, and Java) into machine code (binary: 1s and 0s).

Steps in Compilation:

1. **Writing Code** – The programmer writes source code in a programming language.
2. **Compilation** – The compiler translates the code into machine code.
3. **Linking** – Combines different parts of the program, including libraries, to create the final executable file.
4. **Execution** – The compiled program runs on the computer.

Types of Compilation:

- **Ahead-of-Time (AOT) Compilation:** The entire program is compiled before running (e.g., C, C++).
- **Just-in-Time (JIT) Compilation:** The code is compiled while the program is running (e.g., Java, C#).

- **Interpreted Languages** : Some languages (e.g., Python, JavaScript) do not need compilation but instead use an interpreter to execute code line by line.

Compiling helps make programs faster and more efficient by converting them into a form the computer can understand directly.

Question no: 2

Differentiate between project and solution?

In software development, especially in **Visual Studio**, the terms **Project** and **Solution** have distinct roles in organizing and managing code.

Project:

- A **project** is a single software application or component being developed.
- It includes **source code, resources, dependencies, and configuration files**.
- A project compiles into an **executable file (.exe), a library (.dll), or another output type**.
- Examples of projects include a **console application, a web application, or a class library**.
- Common project file extensions: **.csproj (C#), .vcxproj (C++), .vbproj (VB)**.

Solution:

- A **solution** is a container that organizes multiple related projects.
- It helps in managing large software systems where multiple projects work together.
- The solution itself **does not compile** but groups different projects for development and execution.
- A solution is represented by a **.sln (solution) file**.

Example Scenario:

Imagine developing a **banking system**:

- **Projects in the solution:**
 - **Frontend** (React or ASP.NET Web App)
 - **Backend API** (C# .NET Core)
 - **Database Service** (SQL scripts, database utilities)
 - **Common Library** (Shared utilities for frontend & backend)
- The **solution** groups all these projects so they can be managed together.

Thus, a **project is a single unit of development**, while a **solution is a collection of multiple projects** working together.

Question no:3

What are the three steps for making VB applications?

Three Steps for Making a VB (Visual Basic) Application

Creating a **Visual Basic (VB) application** involves three main steps:

1. Designing the User Interface (UI)

- Open **Visual Studio** and create a new **VB Windows Forms Application**.
- Use the **Toolbox** to **drag and drop controls** (buttons, labels, textboxes, etc.) onto the form.
- Adjust properties like **size, color, font, and name** using the **Properties Window**.

2. Writing the Code

- Double-click on controls (e.g., a button) to open the **Code Editor**.
- Write **VB code** to define the functionality of the application.

3. Running and Testing the Application

- Click the **Start (▶) button** or press **F5** to **run** the application.
- Test the app by **interacting** with buttons, text boxes, etc.
- If needed, **debug and fix errors** using the **Error List** and **Break points**.

After testing, you can **build** and **publish** the application for others to use.

Question no :4

Differentiate between Text property and Name property?

In **VB.NET**, controls have various properties that define their appearance and behavior. Two important properties are **Text** and **Name**.

1. Text Property

The **Text property** is used to define the text that appears on a control in a **Visual Basic (VB) application**.

Features of the Text Property:

- Determines what **text is displayed** on the control.
- It is **visible to the user** in the application's interface.
- Can be modified at **design time** (using the **Properties Window**) or at **runtime** using code

2. Name Property

The **Name property** is used to **identify a control** in the application's code. It serves as a unique identifier for each control.

Features of the Name Property:

- Used to **refer to the control in the code** (e.g., event handlers, functions).
- It is **not visible to the user** on the interface.
- Set during **design time** and does **not change at runtime**.
- Naming convention: Typically starts with a **prefix** (e.g., `btn` for Button, `lbl` for Label, `txt` for TextBox) to improve readability.

Question no: 5

What is identifier?

An **identifier** is the **name** used to identify variables, functions, classes, objects, and other programming elements in a program. It is a unique name assigned by the programmer to refer to a specific entity in the code.

Key Features of Identifiers:

- ✓ Used to **name variables, constants, functions, classes, objects, arrays, etc.**
- ✓ Must start with a **letter (A-Z or a-z) or an underscore (_)**.
- ✓ Can contain **letters, digits (0-9), and underscores (_)** but no spaces or special characters.
- ✓ **Cannot be a keyword** (e.g., `Dim`, `Sub`, `If` in VB.NET).
- ✓ **Case-sensitive** in some languages (not in VB.NET).

Rules for Naming Identifiers in VB.NET:

- ✓ Must start with a **letter or an underscore (_)**.
- ✓ Cannot contain **spaces, special characters, or reserved keywords**.
- ✓ Should be **meaningful and readable** (avoid names like `x`, `y`).
- ✓ **CamelCase or PascalCase** is commonly used (`firstName`, `StudentAge`).

✓ Valid Identifiers:

- `User Name`
- `Total Amount`
- `_temp Value`

- Employee ID

✗ Invalid Identifiers:

- 2ndValue (Cannot start with a digit)
- first name (Cannot contain spaces)
- Dim (Cannot be a reserved keyword)

Conclusion:

An **identifier** is simply the **name** given to variables, functions, and other elements in a program to make them **easy to reference and manage**. Using clear and meaningful identifiers improves **code readability and maintainability**.

Question no: 6

What are multidimensional arrays?

A **multidimensional array** is an array with more than one dimension, meaning it consists of multiple rows and columns. Unlike single-dimensional arrays that store values in a **linear format**, multidimensional arrays store data in a **tabular (2D), cubic (3D), or higher-dimensional structure**.

Types of Multidimensional Arrays

1.

Two-Dimensional (2D) Arrays

2.

1. This is the most common type of multidimensional array, often referred to as a **matrix** or **table**.
2. It consists of **rows and columns**, where each element is accessed using two indices.
3. Commonly used in applications involving grids, spreadsheets, or tabular data storage.

3.

Three-Dimensional (3D) Arrays

4.
 1. A **three-dimensional array** extends the concept of a 2D array by adding another dimension, which can be thought of as **multiple layers of tables**.
 2. It is useful for handling **complex data structures** like 3D graphics, simulations, or layered datasets.
 3. Each element in a 3D array is accessed using three indices: **depth, row, and column**.
- 5.

Higher-Dimensional Arrays

6.
 1. Arrays can have more than three dimensions, but they become increasingly complex to manage.
 2. Used in **scientific computing, game development, and advanced data modeling**.
 3. Higher-dimensional arrays are typically used when dealing with large-scale multi-layered data structures.

Characteristics of Multidimensional Arrays

- ✓ **Stores data in a structured way** using multiple indices.
 - ✓ **Each dimension is indexed separately**, starting from zero.
 - ✓ **Efficient for mathematical computations** involving matrices and tables.
 - ✓ **Requires more memory compared to single-dimensional arrays**, as the storage size increases exponentially with additional dimensions.
 - ✓ **Access time increases** with the number of dimensions, requiring optimized loops for handling large data sets.
-

Advantages of Multidimensional Arrays

Organized Data Representation – Helps store and manage data efficiently in **tabular or layered format**.

Better Performance in Complex Operations – Useful in **matrix operations, simulations, and game programming**.

Suitable for Large-Scale Applications – Used in **image processing, databases, and data analytics**.

Disadvantages of Multidimensional Arrays

✗ **Increased Memory Usage** – More dimensions require significantly more memory.

✗ **Complex Indexing** – Managing multiple indices can be challenging in large applications.

✗ **Difficult Debugging** – Errors related to array indexing can be harder to detect in multidimensional structures.

Applications of Multidimensional Arrays

Storing tabular data like spreadsheets, matrices, and databases.

Game development for handling grid-based maps or objects.

Scientific computations involving mathematical modeling and physics simulations.

Image processing where pixel data is stored in a 2D or 3D array.

Artificial Intelligence and Machine Learning for handling complex data structures.

Question no:7

What is rational expressions?

In **computer science and programming**, a **rational expression** refers to a mathematical expression in the form of a **fraction**,

where both the numerator and denominator are **polynomials**. These expressions are commonly used in **computational mathematics, algorithms, and programming applications** that involve fractions, algebraic operations, or symbolic computations.

Where Are Rational Expressions Used in Computing?

1. Mathematical Computation and Symbolic Algebra

- **Computer Algebra Systems (CAS)** like MATLAB, Wolfram Mathematica, and SymPy in Python can process rational expressions symbolically.
- Used in **scientific computing** to solve equations involving fractions of polynomials.

2. Programming Languages and Software Development

- Some programming languages support **fractions and rational numbers** as data types (e.g., Python's `fractions` module).
- Rational expressions help in **precision calculations** where floating-point arithmetic can introduce errors.

3. Computer Graphics and Simulations

- Used in **game physics, rendering engines, and simulations** to model transformations, scaling, and animations.
- Helps in **interpolation and curve fitting** for smooth motion in graphics.

4. Cryptography and Data Encryption

- **Number theory** in cryptographic algorithms often involves rational expressions.

- **Elliptic curve cryptography (ECC)** relies on rational operations over finite fields.

5. Artificial Intelligence and Machine Learning

- Used in **optimization problems**, especially in gradient descent calculations.
- Applied in **symbolic AI** for manipulating algebraic expressions.

6. Networking and Data Processing

- Rational expressions may be used in **error detection and correction algorithms** in data transmission.
- Used in **packet scheduling algorithms** to balance network loads efficiently.

Question no:8

how does an array differ from simple variable?

In programming, both **arrays** and **simple variables** are used to store data, but they have **key differences** in terms of storage, structure, and functionality.

Feature	Simple Variable	Array
Definition	Stores a single value at a time.	Stores multiple values of the same data type.
Size	Holds only one value at a time.	Can hold multiple values (fixed or dynamic size).
Data Type	Stores a single value of a specific data type (e.g., int, string).	Stores multiple values of the same data type.

Feature	Simple Variable	Array
Access	Accessed directly by the variable name.	Accessed using an index (e.g., array[0], array[1]).
Memory Usage	Requires less memory , as it holds only one value.	Requires more memory , as it stores multiple values in contiguous locations.
Usage	Used for storing a single value (e.g., a person's age, a single character).	Used for storing multiple related values (e.g., a list of student grades, names).
Modification	Holds a single value , so modification affects only one data item.	Can modify individual elements without affecting others.
Looping	No need for loops, as it stores only one value.	Can be iterated using loops (e.g., For loop) for efficient processing.

Conclusion

- A **simple variable** stores **only one value** at a time, while an **array** stores **multiple values** of the same type.
- Arrays are **more efficient** for handling multiple values but require **indexing** to access elements.
- Arrays are ideal for storing **lists, collections, or sequences** of data, while simple variables are best for storing **single values**.

Question no:9

What is modality?

In **computing**, **modality** refers to how a system, application, or interface **restricts user interaction** based on certain conditions or states. It primarily applies to **user interface (UI) design**, human-computer interaction, and **artificial intelligence (AI)**.

1. Modality in User Interfaces (UI/UX)

- In **UI design**, **modality** refers to the use of **modes** that require user input before proceeding.
- A **modal window/dialog box** interrupts the main workflow and forces the user to complete an action before continuing.

Examples:

- ✓ **Pop-up Message or Alert** – A confirmation dialog (e.g., "Are you sure you want to delete this file?").
- ✓ **Login Screen** – Prevents access to an application until login is completed.
- ✓ **Full-Screen Mode** – In video players, where other controls are hidden.

Types of Modality in UI:

Modal – Requires user interaction before proceeding. (*Example: Confirmation pop-ups, Login screens*)

Non-Modal – Allows the user to continue working without interruption. (*Example: Sidebars, tool palettes in Photoshop*)

2. Modality in Artificial Intelligence (AI) and Machine Learning (ML)

- In **AI and ML**, **modality** refers to the **type of data input** used by a system.
- AI systems often work with **multiple modalities** (multimodal learning) to improve accuracy.

Common AI Modalities:

Text Modality – Used in chatbots, NLP (e.g., ChatGPT).

Image Modality – Used in facial recognition, object detection.

Audio/Speech Modality – Used in voice assistants like Siri, Alexa.

Video Modality – Used in video analytics, surveillance.

3. Modality in Programming and Systems Design

- **Modal Programming** refers to applications that switch between different states or modes.
- **Modes can change functionality**, affecting how users interact with software.

Examples:

Word Processors – "Edit Mode" vs. "Read-Only Mode".

Video Games – "Pause Mode" where normal gameplay is suspended.

4. Modality in Virtual Reality (VR) & Augmented Reality (AR)

- In **VR/AR**, modality refers to **how users interact with a virtual environment** using different input methods.
 - **Examples:**
 - Hand gestures (Gesture Modality).
 - Voice commands (Speech Modality).
 - Eye tracking (Gaze Modality).
-

Question no :10

how does a check box differ from radio buttons?

Both **Check Boxes** and **Radio Buttons** are commonly used UI elements in forms, but they serve different purposes in terms of selection behavior.

Feature	Check Box	Radio Button
Selection	Allows selecting	Allows selecting only

Feature	Check Box	Radio Button
Type	multiple options.	one option at a time within a group.
Behavior	Each check box operates independently.	Works as a group , where selecting one option automatically deselects others.
Use Case	Used when multiple choices are allowed (e.g., selecting multiple interests).	Used when only one choice is required (e.g., selecting gender).
Appearance	Small square box that can be checked or unchecked.	Small circular button , with a filled dot when selected .
Default State	Can be checked or unchecked by default.	One option is often pre-selected by default.
Example	"Select the fruits you like: " (Can select multiple).	"Select your gender: ☐ Male ☐ Female" (Can select only one).

Key Differences:

- ✓ **Check Box** – Used for **multiple selections**.
- ✓ **Radio Button** – Used for **single selections** within a group.

Conclusion:

Use **check boxes** when users need to select **multiple options** and **radio buttons** when they must pick **only one choice** from a set.